Data Models

SQL Querying

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

Building On: Topics 2–3

Topics 2 and 3 produced a **structurally correct, normalised schema** — ERD built, BCNF verified, star schema designed for reporting.

The schema is sound. The question shifts: **how do you retrieve the right data from this shape?**

In Topic 3's lab, you ran a 5-table reporting query on faith — you could observe the result but not explain the mechanics. This chapter gives you the vocabulary to explain it fully.

Four Failures, One Sprint

The team writes queries against the BCNF schema. In a single sprint, four queries produce four kinds of wrong answer.

What makes these worth studying: **none of them throw errors.** The SQL runs. The database returns a result set. The numbers look reasonable. Nobody questions them — until the VP reads the report.

All four trace to one root cause: the developers have no mental model of how SQL processes rows.

The Sample Dataset

Four tables, seventeen rows — small enough to trace every failure by hand.

```
oustomers
customer_id | name        | country
------------+-------------+--------
1           | Alice Tan   | SG
2           | Bob Patel   | MY
5           | Eve Santos  | SG

orders
order_id | customer_id | order_date | status    | order_total
---------+-------------+------------+-----------+------------
101      | 1           | 2024-03-15 | completed | 34.97
102      | 1           | 2024-06-20 | completed | 49.99
103      | 2           | 2024-04-10 | shipped   | 24.98
104      | 2           | 2024-09-01 | paid      | 49.99
```

The Sample Dataset — Line Items and Products

```
order_lines
line_id | order_id | product_id | quantity | unit_price
────────┼──────────┼────────────┼──────────┼───────────
1       | 101      | 1          | 2        | 9.99
2       | 101      | 2          | 1        | 14.99
3       | 102      | 3          | 1        | 49.99
4       | 103      | 1          | 1        | 9.99
5       | 103      | 2          | 1        | 14.99
6       | 104      | 3          | 1        | 49.99

products
product_id | name        | unit_price
───────────┼─────────────┼──────────
1          | Widget A    | 9.99
2          | Widget B    | 14.99
3          | Gadget X    | 49.99
6          | Screen 24"  | 199.99
```

Schema Map — FK Arrows and Join Direction

```
E-commerce tables:

  customers ←──FK── orders ←──FK── order_lines ──FK──→ products
    (3 rows)         (4 rows)        (6 rows)            (4 rows)

  Join direction (follow the arrow):
    customers → orders          PK → FK   one-to-many (multiplies)
    orders → order_lines        PK → FK   one-to-many (multiplies)
    order_lines → products      FK → PK   many-to-one (preserves)
    customers → orders → order_lines      multiplies twice
```

The prediction skill: **FK → PK** (many-to-one) preserves row count; **PK → FK** (one-to-many) multiplies rows.

The chain customers → orders → order_lines multiplies twice — this is the root of Failure 1.

Failure 1 — Count Inflation

```
SELECT c.name, COUNT(*) AS order_count
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN order_lines ol ON o.order_id = ol.order_id
GROUP BY c.customer_id, c.name;
```

Alice shows **3 orders instead of 2**. The join to `order_lines` multiplied rows before `COUNT` saw them — Alice has 2 orders but 3 line items.

`COUNT(*)` counted line items, not orders. At production scale — a 2-order customer can show 50 or more.

The developer didn't predict the row multiplication.

Failure 2 — Silent Data Loss

```
SELECT p.name, SUM(ol.quantity) AS units_sold
FROM products p
INNER JOIN order_lines ol ON p.product_id = ol.product_id
GROUP BY p.product_id, p.name;
```

The catalogue has 4 products. The query returns **3**. Screen 24" — the product with zero sales — vanished silently.

INNER JOIN drops rows that have no match. No error. No warning. The report looked plausible, so nobody noticed.

The only way to catch this: know, before running the query, that INNER JOIN drops unmatched rows.

Failure 3 — Aggregate in WHERE

```
SELECT c.name, SUM(ol.quantity * ol.unit_price) AS total_spend
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN order_lines ol ON o.order_id = ol.order_id
WHERE SUM(ol.quantity * ol.unit_price) > 100
GROUP BY c.customer_id, c.name;
```

Syntax error. The analyst moves the `SUM` to SELECT — the query runs, but now returns every customer, not just big spenders.

Every attempt to fix it is trial and error because the analyst has no model of *when* each clause executes. `WHERE` runs before `GROUP BY` creates groups — there is nothing to `SUM` over at that point.

Failure 4 — Duplicated Subquery Drift

The analyst needs "customers whose total spend exceeds the average customer spend." This requires two passes over the data — they copy-paste the same subquery in two places.

When the business logic changes (exclude refunded orders), they update **one copy** and forget the other.

The results silently diverge. A customer appears in the "above average" list even though their spend is below the average shown in the same report.

Root cause: the same computation expressed in two places, with one copy drifting.

Four Failures — What Do They Share?

	Symptom
Failure 1	Alice shows 3 orders instead of 2
Failure 2	Screen 24" vanishes from the report
Failure 3	Syntax error → wrong fix → wrong results
Failure 4	Report contradicts itself

None threw an error. All returned plausible results. All went to the VP unquestioned.

> **Predict:** These are four different queries with four different symptoms. What single gap, shared by all four developers, made every failure possible?

Four Failures — Common Root

> **No mental model of how SQL processes rows.** The schema is correct. The SQL syntax is (mostly) correct. What is missing is a way to trace, stage by stage, what the query engine does to the row set.

	Symptom	Root cause
Failure 1	Alice shows 3 orders instead of 2	Join multiplied rows before COUNT
Failure 2	Screen 24" vanishes from the report	INNER JOIN silently drops unmatched rows
Failure 3	Syntax error → wrong fix → wrong results	No model of when each clause executes
Failure 4	Report contradicts itself	Same logic in two places; one drifts

Therefore...

> Therefore, the query author must **trace what the engine does to the row set at each stage** — not guess from syntax, not debug from wrong output, not copy-paste from Stack Overflow.

Four capabilities define what "trace" means:

1. **Row-set traceability** — predict the row set after each clause before running it
2. **Join multiplicity awareness** — predict when joins multiply, preserve, or eliminate rows
3. **Aggregation placement rules** — know which clauses can contain aggregates, derived from execution order
4. **Composability** — express multi-step logic with named intermediate results, without duplication

Four Capabilities — Mapped to Failures

Capability	Failure prevented	Test
Row-set traceability	Failure 2: Screen 24" vanished	Predict which rows survive each clause
Join multiplicity awareness	Failure 1: Alice shows 3 orders	Predict the row count after each join
Aggregation placement rules	Failure 3: SUM in WHERE	Name which clause runs before GROUP BY
Composability	Failure 4: subquery drift	Express intermediate results once, reference many times

All four derive from a single mental model — a procedural trace of what the query engine does, stage by stage, to a set of rows.

The Mental Model — Three Components

Component	What it provides	Failures it diagnoses
For-Each model	Procedural trace of joins — nested loops over rows	Failure 1 (count inflation), Failure 2 (silent data loss)
FWGHOS pipeline	Fixed logical execution order of SQL clauses	Failure 3 (aggregate in WHERE)
CTE (Common Table Expression)	Named intermediate results — compute once, reference anywhere	Failure 4 (subquery drift)

The rest of this topic builds these three components, one at a time.

The For-Each Model — Single Tables

The simplest mental model for SQL: imagine a **For-Each loop** over the rows of a table.

```
SELECT name, country
FROM customers
WHERE country = 'SG';
```

```
FOR EACH row r IN customers:
    IF r.country = 'SG':
        OUTPUT (r.name, r.country)
```

Query Output:

name	country
Alice Tan	SG
Eve Santos	SG

```
customers
customer_id | name        | country
------------+-------------+--------
1           | Alice Tan   | SG
2           | Bob Patel   | MY
5           | Eve Santos  | SG
```

The pseudocode and the SQL describe the same operation. This explains what `SELECT` does (pick columns), what `WHERE` does (filter rows), and why duplicates appear (two rows with the same values produce two output rows).

Nested Loop — Inner Joins

The For-Each model extends to joins. `FROM A JOIN B ON A.id = B.a_id` is a **nested For-Each**:

```
FOR EACH row a IN A:
    FOR EACH row b IN B:
        IF a.id = b.a_id:
            OUTPUT (a.*, b.*)
```

Three consequences:

Match Type	What Happens	Row-Count Effect
One-to-one	1 row in A matches 1 in B	Preserves
One-to-many	1 row in A matches N in B	**Multiplies** (Failure 1)
No match	A row matches 0 in B	**Drops silently** (Failure 2)

Three Join Consequences: Visualised

```
One-to-one:       A ─── B          → 1 output row
                  (1 match)

One-to-many:      A ─┬─ B₁         → 3 output rows
                     ├─ B₂           (row multiplication)
                     └─ B₃

No match:         A ─── ∅          → 0 output rows
                  (INNER JOIN       (silent data loss)
                   drops the row)
```

The prediction skill: before running a join, ask — **for each row on the left, how many matches exist on the right?**

Join Direction Predicts Row Count

Join Path	Direction	Row-Count Effect	Example
FK → PK (many-to-one)	orders → customers	**Preserves:** each order matches exactly one customer	4 orders → 4 rows
PK → FK (one-to-many)	customers → orders	**Multiplies:** each customer matches N orders	3 customers → 4 rows
FK → PK → FK (chain)	order_lines → orders → customers	Preserves twice	6 lines → 6 rows
PK → FK → FK (chain)	customers → orders → order_lines	Multiplies twice	3 customers → 6 rows
Junction (many-to-many)	students → enrolments → courses	Multiplies both sides	Row count = enrolment count

Predict: `orders` has 4 rows. You join `orders` to `order_lines` on `order_id`. Is the join one-to-one or one-to-many? What is the row count?

Join Direction — Revealed

> **One-to-many.** Each order can have multiple line items — order 101 has 2 lines, order 103 has 2 lines. The join multiplies: 4 orders become **6 rows**.

This is exactly the multiplication that caused Failure 1. The developer joined customers → orders → order_lines (PK → FK twice) and assumed the row count would stay at one per customer. It didn't.

Cost of the prediction skill: a few seconds of thinking before running. **Benefit:** catching wrong results before the VP reads the report.

The Sales-Organisation Dataset

A second dataset — self-referencing employees and offices:

```
employees
employee_id | first_name | last_name | office_id | salary   | manager_id
————————————|————————————|———————————|———————————|——————————|————————————
1           | Sarah      | Tan       | 1         | 85,000   | NULL
2           | Wei Jie    | Lim       | 1         | 52,000   | 1
3           | Budi       | Santoso   | 4         | 55,000   | 1
4           | Ahmad      | Ismail    | 2         | 34,000   | 2
7           | Rizal      | Pratama   | 4         | 56,000   | 3

products
product_id | name       | unit_price
————————————|————————————|————————————
1           | Widget A   | 9.99
2           | Widget B   | 14.99
3           | Gadget X   | 49.99
6           | Screen 24" | 199.99
```

Sarah Tan: `manager_id = NULL` (managing director). Ho Chi Minh City: 0 employees.

Self-Join — Same Table on Both Sides

"List each employee with their manager's name." Both are in `employees`. A single For-Each loop visits each employee row — but how do you access the *manager's* row during that visit?

```
FOR EACH row e IN employees:
    FOR EACH row m IN employees:
        IF e.manager_id = m.employee_id:
            OUTPUT (e.first_name, e.last_name,
                    m.first_name, m.last_name)
```

```
SELECT e.first_name AS employee_first, e.last_name AS employee_last,
       m.first_name AS manager_first, m.last_name AS manager_last
FROM employees e
INNER JOIN employees m ON e.manager_id = m.employee_id;
```

A self-join is an ordinary inner join where both sides happen to be the **same table**.

Self-Join Trace

```
e.employee_id  e.first_name  e.manager_id → m.employee_id  m.first_name
    2          Wei Jie            1              1           Sarah
    3          Budi               1              1           Sarah
    4          Ahmad              2              2           Wei Jie
    7          Rizal              3              3           Budi
    — Sarah (manager_id = NULL) → no match → dropped by INNER JOIN
```

5 employees → **4 rows.** Sarah Tan is the managing director — her `manager_id` is NULL. The inner loop finds no match (`NULL` matches nobody), so INNER JOIN drops her.

> **Predict:** To keep Sarah in the result, what must change?

LEFT JOIN — One Rule Change

LEFT JOIN changes one rule in the nested loop: when a row in A matches **0 rows** in B, instead of dropping it, **keep it and fill B's columns with NULL**.

```
SELECT e.first_name AS employee_first, e.last_name AS employee_last,
       m.first_name AS manager_first, m.last_name AS manager_last
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.employee_id;
```

```
INNER JOIN (4 rows):                    LEFT JOIN (5 rows):
employee    manager                     employee    manager
____________ ____________               ____________ ____________

Wei Jie Lim   Sarah Tan                Wei Jie Lim   Sarah Tan
Budi Santoso  Sarah Tan                Budi Santoso  Sarah Tan
Ahmad Ismail  Wei Jie Lim              Ahmad Ismail  Wei Jie Lim
Rizal Pratama Budi Santoso             Rizal Pratama Budi Santoso
                                       Sarah Tan     NULL  NULL
              ↑ Sarah dropped                        ↑ Sarah kept
```

NULL Means Unknown — Two Traps

NULL is not zero, not empty, not false. NULL means **unknown**. Any comparison with NULL (including `NULL = NULL`) evaluates to UNKNOWN — which `WHERE` treats as false.

Trap 1 — WHERE undoes LEFT JOIN. `WHERE m.last_name = 'Tan'` after a LEFT JOIN silently drops all unmatched rows (where `m.last_name` is NULL), converting the LEFT JOIN back to an effective INNER JOIN. **Fix:** put right-table filters in the **ON clause**, not WHERE.

Trap 2 — Equality doesn't work with NULL. `WHERE column = NULL` returns zero rows, even when NULLs exist. **Fix:** use `IS NULL` / `IS NOT NULL`.

Both traps share one root: NULL is **not a value** — it is the absence of a value. Standard comparisons (`=`, `<>`, `<`, `>`) with NULL always return UNKNOWN.

COUNT Behaviour with NULLs

Predict: `offices` LEFT JOIN `employees` on office_id. Ho Chi Minh City (office 5) has no employees. What does `COUNT(*)` return for that office? What does `COUNT(e.employee_id)` return?

COUNT(*) vs COUNT(column) — Revealed

```
office_id  city              e.employee_id   COUNT(*)  COUNT(e.employee_id)
    1      Singapore              1              —             —
    1      Singapore              2              —             —
    5      Ho Chi Minh City      NULL            —             —

After GROUP BY office_id:
    office 1 (Singapore):        COUNT(*) = 2    COUNT(e.employee_id) = 2
    office 5 (Ho Chi Minh City): COUNT(*) = 1    COUNT(e.employee_id) = 0
                                      ↑                    ↑
                               NULL-padded row     NULL skipped
                               still counted       correctly
```

> `COUNT(*)` counts **all rows** including NULL-padded ones. `COUNT(column)` **skips NULLs**. This distinction is the difference between an office showing headcount 1 vs 0 when it has no employees.

Left Anti-Join — Finding Unmatched Rows

Failure 2 showed that INNER JOIN *accidentally* drops unmatched rows. The left anti-join **deliberately** finds rows with no match:

```
SELECT p.name
FROM products p
LEFT JOIN order_lines ol ON p.product_id = ol.product_id
WHERE ol.product_id IS NULL;
```

```
Step 1 — LEFT JOIN (keep all products, 7 rows):
    product_id  name        ol.product_id
         1      Widget A         1          ← matched (line 1)
         1      Widget A         1          ← matched (line 4)
         2      Widget B         2          ← matched (line 2)
         2      Widget B         2          ← matched (line 5)
         3      Gadget X         3          ← matched (line 3)
         3      Gadget X         3          ← matched (line 6)
         6      Screen 24"      NULL        ← no match → NULL-padded

Step 2 — WHERE ol.product_id IS NULL:
    product_id  name
         6      Screen 24"                  ← the only unmatched product
```

NOT IN vs Left Anti-Join — The NULL Trap

The alternative to the left anti-join:

```
SELECT name FROM products
WHERE product_id NOT IN (SELECT product_id FROM order_lines);
```

Both produce the same result — *unless* the subquery returns a NULL.

Predict: If you insert a row into `order_lines` with `product_id = NULL`, what does the `NOT IN` version return? What does the left anti-join version return?

NOT IN Trap — Revealed

> `NOT IN` returns **zero rows** — the NULL poisons the entire expression (`x ≠ NULL` is UNKNOWN for every x). The left anti-join still correctly returns Screen 24" — `IS NULL` handles NULLs by design.

Cost: the anti-join requires more syntax. **Benefit:** it is the safer default.

The Execution Pipeline — FWGHOS

SQL clauses execute in a fixed logical order, regardless of how you write the query. The mnemonic is **FWGHOS** (pronounced "few-goes"):

Step	Clause	What it does
1 — **F**	FROM / JOIN	Assemble the row set
2 — **W**	WHERE	Filter individual rows
3 — **G**	GROUP BY	Partition into groups
4 — **H**	HAVING	Filter groups
5 — **S**	SELECT	Choose columns, compute expressions
6 — **O**	ORDER BY	Sort result

Every clause takes a row set as input and produces a row set as output. The pipeline is a chain of transformations.

(The mnemonic reverses S and O for pronounceability; real execution order is SELECT at step 5, then ORDER BY at step 6, as the table shows.)

Three Errors the Pipeline Explains

Error	Why It Fails	FWGHOS Explanation
SELECT alias in WHERE	`WHERE total_spend > 100`	SELECT runs at step 5; WHERE runs at step 2 — alias doesn't exist yet
Aggregate in WHERE	`WHERE SUM( ... ) > 100`	Groups don't exist at step 2; GROUP BY creates them at step 3
SELECT alias in ORDER BY	`ORDER BY total_spend`	**Works** — ORDER BY runs at step 6, after SELECT (step 5)

These are not rules to memorise — they are **consequences of the pipeline order**. If you know FWGHOS, you can derive them.

Predict: A colleague writes `HAVING total_spend > 100` where `total_spend` is a SELECT alias. Does this work in standard SQL?

HAVING Alias — Revealed

> **In standard SQL, no.** HAVING runs at step 4, before SELECT at step 5 — the alias doesn't exist yet. Repeat the expression: `HAVING SUM(ol.quantity * ol.unit_price) > 100`.

MySQL accepts the alias as an extension. Developers who learn on MySQL and switch to PostgreSQL get an error — the FWGHOS pipeline makes this predictable.

DISTINCT and COUNT(DISTINCT)

DISTINCT eliminates duplicate rows *after* SELECT. It is the right tool when your projection naturally creates duplicates — e.g., `SELECT DISTINCT city FROM customers`.

But DISTINCT is **not a fix for a bad join**. If your join inflated rows, DISTINCT might hide the symptom but won't fix the cause.

COUNT(DISTINCT column) solves a specific problem: after a join that introduces multiplicity, count **unique values** rather than inflated rows.

> **Predict:** In Failure 1, would adding DISTINCT to the SELECT fix the inflated count?

COUNT(DISTINCT) — Revealed

> **No.** DISTINCT eliminates duplicate *output rows*, but each row already has a unique combination of (name, count). The count itself is wrong — it counted line items, not orders.

The fix is `COUNT(DISTINCT o.order_id)` — count unique orders within each group regardless of how many line items they contributed:

```
SELECT c.name, COUNT(DISTINCT o.order_id) AS order_count
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN order_lines ol ON o.order_id = ol.order_id
GROUP BY c.customer_id, c.name;
```

Alice now shows **2 orders** — correct.

CTEs — Named Intermediate Results

Failure 4's root cause: the same computation in two places, with one copy drifting. A **CTE (Common Table Expression)** is a named, temporary result set — the query equivalent of a local variable.

```
WITH customer_spend AS (
    SELECT c.customer_id, c.name,
           SUM(ol.quantity * ol.unit_price) AS total_spend
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN order_lines ol ON o.order_id = ol.order_id
    GROUP BY c.customer_id, c.name
)
SELECT name, total_spend
FROM customer_spend
WHERE total_spend > (SELECT AVG(total_spend) FROM customer_spend);
```

`customer_spend` defined **once**, referenced **twice**. When the logic changes, one edit propagates everywhere.

Cost: a CTE is not faster than a subquery — the engine may execute it identically. **Benefit:** maintainability — one definition, one place to update.

CTE Trace

```
WITH customer_spend AS (
    customer_id  name        total_spend
         1       Alice Tan    84.96      ← (9.99×2) + 14.99 + 49.99
         2       Bob Patel    74.97      ← 9.99 + 14.99 + 49.99
)

AVG(total_spend) = (84.96 + 74.97) / 2 = 79.965

SELECT: Alice (84.96 > 79.965) → output
        Bob   (74.97 < 79.965) → filtered out

Result: Alice Tan, 84.96
```

Without the CTE, the same subquery appears in two places. Change the business rule (exclude refunded orders): **CTE = 1 edit. Duplicated subqueries = 2 edits.** Forgetting one is exactly Failure 4.

This is Topic 3's "single source of truth" applied to queries rather than schemas.

End-to-End FWGHOS Trace — Setup

The chapter's centrepiece. The corrected version of Failure 1, traced stage by stage.

Business question: "For each customer, show the number of orders and total spend, but only for customers with 2+ orders on or after 1 April 2024. Sort by total spend descending."

```
SELECT c.name,
       COUNT(DISTINCT o.order_id) AS order_count,
       SUM(ol.quantity * ol.unit_price) AS total_spend
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id
INNER JOIN order_lines ol ON o.order_id = ol.order_id
WHERE o.order_date ≥ '2024-04-01'
GROUP BY c.customer_id, c.name
HAVING COUNT(DISTINCT o.order_id) ≥ 2
ORDER BY total_spend DESC;
```

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│ F: FROM      │ ──→ │ W: WHERE     │ ──→ │ G: GROUP BY  │ ──→ │ H: HAVING    │ ──→ │ S: SELECT    │ ──→ │O: ORDER BY   │
│ 6 rows       │    │ 4 rows       │    │ 2 groups     │    │ 1 group      │    │ 1 row        │    │ 1 row        │
└──────────────┘    └──────────────┘    └──────────────┘    └──────────────┘    └──────────────┘    └──────────────┘
```

Six stages, 6 rows narrowed to 1 output row. Let's trace each stage with actual row data.

Step F — FROM / JOIN

`customers ⋈ orders ⋈ order_lines` — one row per line item:

```
customer_id  name       order_id  order_date   line_id  product_id  quantity
unit_price
    1        Alice Tan     101    2024-03-15       1         1          2       9.99
    1        Alice Tan     101    2024-03-15       2         2          1      14.99
    1        Alice Tan     102    2024-06-20       3         3          1      49.99
    2        Bob Patel     103    2024-04-10       4         1          1       9.99
    2        Bob Patel     103    2024-04-10       5         2          1      14.99
    2        Bob Patel     104    2024-09-01       6         3          1      49.99
```

6 rows. Eve (customer 5, 0 orders) dropped by INNER JOIN. Alice: 3 rows (2 orders, 3 lines). Bob: 3 rows (2 orders, 3 lines). This is where Failure 1's row multiplication happens.

Step W — WHERE + Step G — GROUP BY

W: `WHERE o.order_date >= '2024-04-01'` removes Alice's order 101 (2024-03-15) — 2 rows dropped. **4 rows survive.**

customer_id	name	order_id	order_date	line_id	quantity	unit_price
1	Alice Tan	102	2024-06-20	3	1	49.99
2	Bob Patel	103	2024-04-10	4	1	9.99
2	Bob Patel	103	2024-04-10	5	1	14.99
2	Bob Patel	104	2024-09-01	6	1	49.99

G: `GROUP BY c.customer_id, c.name` creates **2 groups:**

- Alice Tan: 1 row (1 order after the date filter)
- Bob Patel: 3 rows (2 orders, 3 line items)

Step H — HAVING + Step S — SELECT + Step O — ORDER BY

H: `HAVING COUNT(DISTINCT o.order_id) >= 2`

- Alice: COUNT(DISTINCT order_id) = **1** (only order 102) → **dropped**
- Bob: COUNT(DISTINCT order_id) = **2** (orders 103, 104) → **kept**

1 group survives.

S: For Bob's group: `name` = 'Bob Patel', `order_count` = 2, `total_spend` = 9.99 + 14.99 + 49.99 = **74.97**

O: 1 row — sorting is trivial.

```
F: 6 rows (Eve dropped by INNER JOIN)
  → W: 4 rows (Alice's order 101 filtered by date)
    → G: 2 groups (Alice: 1 row; Bob: 3 rows)
      → H: 1 group (Alice dropped: 1 distinct order < 2)
        → S: Bob Patel | order_count=2 | total_spend=74.97
          → O: 1 row (sorted — trivial)
```

What the Trace Makes Visible

Three insights from the end-to-end trace:

Why COUNT(DISTINCT) matters: Without DISTINCT, `COUNT(order_id)` for Bob's group returns 3 (line items), not 2 (orders). The row multiplication from step F is still present inside the group — COUNT(DISTINCT) corrects for it.

Why the aggregate can't go in WHERE: At step W, groups don't exist yet. `COUNT(DISTINCT o.order_id)` is meaningless before GROUP BY creates the partitions.

Why the date filter belongs in WHERE, not HAVING: The date filter applies to individual rows (step W), not groups (step H). Putting it in HAVING would filter groups *after* aggregation — logically different and almost certainly wrong.

The Predict-First Query Method

A repeatable procedure for building any query:

1. **Identify tables and joins** — draw the join path; predict row count at each join (one-to-one, one-to-many, many-to-many?)

2. **Write WHERE** — filter individual rows (no aggregates here; NULL requires `IS NULL`)

3. **Write GROUP BY + HAVING** — predict group count; aggregate conditions go in HAVING

4. **Write SELECT + ORDER BY** — choose output columns; wrap in CTE if reusing intermediate results

5. **Verify** — run on known data, compare to prediction; if row counts differ, trace FWGHOS to find the divergence

Cost: a few minutes of prediction before running. **Benefit:** wrong results caught before production. No guessing, no trial and error.

Why It Works — and What It Doesn't Solve

What the model provides	What it does not solve
Predicts what a query returns — row set after each stage	**Performance** — indexes, query plans, physical tuning (Topic 5)
For-Each trace — bags of rows, no hidden magic	**Data integrity** — what prevents invalid data from entering (Topic 5)
Join direction → row-count prediction	**Recursive traversal** — "all reports up the chain" (Topic 10: property graphs)
FWGHOS → aggregation placement	

The model is the diagnostic foundation. You can't tune a query you can't trace.

Check 1: Count Inflation (Hospital)

A hospital tracks patient visits. Each visit can have multiple procedures. A developer writes:

```
SELECT p.name, COUNT(*) AS visit_count
FROM patients p
INNER JOIN visits v ON p.patient_id = v.patient_id
INNER JOIN procedures pr ON v.visit_id = pr.visit_id
GROUP BY p.patient_id, p.name;
```

A patient with 2 visits and 5 procedures shows `visit_count = 5`. What went wrong? What is the fix?

> The join to `procedures` is one-to-many — row multiplication before COUNT. Fix: `COUNT(DISTINCT v.visit_id)`. Same pattern as Failure 1: predict join direction, use COUNT(DISTINCT) to count the entity, not the inflated rows.

Check 2: Silent Data Loss (Library)

A library system tracks books and loans. A report needs "all books and their loan count":

```
SELECT b.title, COUNT(l.loan_id) AS loan_count
FROM books b
INNER JOIN loans l ON b.book_id = l.book_id
GROUP BY b.book_id, b.title;
```

The library has 500 books. The report shows 487. What happened to the other 13?

> INNER JOIN dropped 13 books that have **never been loaned** — no matching rows in `loans`. Fix: `LEFT JOIN`. The 13 books appear with `loan_count = 0` (because `COUNT(l.loan_id)` skips NULLs). Same pattern as Failure 2.

Check 3: Aggregate Placement (E-commerce)

An analyst writes:

```
SELECT p.name, SUM(ol.quantity) AS total_units
FROM products p
INNER JOIN order_lines ol ON p.product_id = ol.product_id
WHERE SUM(ol.quantity) > 10
GROUP BY p.product_id, p.name;
```

The query fails. The analyst moves the `SUM` to SELECT — it runs but returns all products. Where should the filter go?

> **HAVING.** WHERE runs at step 2 (before groups exist); HAVING runs at step 4 (after GROUP BY). `HAVING SUM(ol.quantity) > 10` filters groups, not individual rows. FWGHOS makes this predictable — not a syntax rule to memorise.

Check 4: LEFT JOIN Trap (Ride-Sharing)

A ride-sharing company needs "all drivers, including those with no trips this month":

```
SELECT d.name, COUNT(t.trip_id) AS trip_count
FROM drivers d
LEFT JOIN trips t ON d.driver_id = t.driver_id
WHERE t.trip_date ≥ '2024-03-01'
GROUP BY d.driver_id, d.name;
```

Drivers with no March trips disappear from the result. Why?

> The `WHERE` clause evaluates `t.trip_date` — which is NULL for drivers with no trips — as false, silently converting the LEFT JOIN back to an INNER JOIN (Trap 1). Fix: move the date filter to the **ON clause**: `LEFT JOIN trips t ON d.driver_id = t.driver_id AND t.trip_date ≥ '2024-03-01'`.

Check 5: CTE vs Duplication (Pharma)

A pharmaceutical company reports "drugs with sales above the average drug sales." An analyst writes it with two identical subqueries. The business rule changes: exclude recalled drugs. They update one subquery and forget the other.

What is the structural problem? What is the fix?

> Same computation in two places → one drifts when logic changes. Fix: a **CTE** — compute `drug_sales` once, reference twice. When the exclusion rule changes, **one edit** propagates to both references. This is Failure 4's pattern: the CTE is the query-level equivalent of "one fact, one place."

Check 6: Self-Join Transfer (School)

A school database has a `teachers` table: `teacher_id`, `name`, `salary`, `mentor_id` (references another teacher). A report needs "all teachers with their mentor's name":

```
SELECT t.name AS teacher, m.name AS mentor
FROM teachers t
INNER JOIN teachers m ON t.mentor_id = m.teacher_id;
```

The head of department (`mentor_id = NULL`) disappears. What is the fix? Why does the current query fail?

> INNER JOIN drops the head — NULL matches nobody in the inner loop. Fix: `LEFT JOIN teachers m ON t.mentor_id = m.teacher_id`. The head appears with `mentor = NULL`. Same pattern as the employee self-join: NULL in a FK means no match, so INNER JOIN silently drops the row.

Check 7: Cross-Topic, Schema Supports Queries

Topic 2 placed foreign keys; Topic 3 normalised to BCNF. Every `ON c.customer_id = o.customer_id` in this chapter works because Topic 2 placed the FK correctly. Every join-direction prediction is reliable because Topic 3 ensured the cardinalities match.

If the schema had a missing FK or wrong cardinality, what would happen to the queries?

> Joins would silently produce wrong results — a failure that traces back to the **design step**, not the query step. The schema is **query infrastructure**. A correct query on a broken schema is still wrong. Two disciplines, two moments — complementary.

Checks 1–7: The Arc

Check	Domain	What it tests	Lesson
1 Count inflation	Hospital	Detection: spotting multiplied counts	Predict join direction, use COUNT(DISTINCT)
2 Silent data loss	Library	Detection: spotting missing rows	LEFT JOIN preserves unmatched rows
3 Aggregate placement	E-commerce	Application: using FWGHOS to place filters	HAVING for group filters; WHERE for row filters
4 LEFT JOIN trap	Ride-sharing	Judgement: WHERE undoes LEFT JOIN	Right-table filters belong in ON, not WHERE
5 CTE vs duplication	Pharma	Transfer: applying composability	CTE = one fact, one place for queries
6 Self-join transfer	School	Transfer: self-join + NULL awareness	LEFT JOIN for self-referencing NULL FKs
7 Schema supports queries	Cross-topic	Boundary: where querying stops	Schema is query infrastructure

Progression: **detection → detection → application → judgement → transfer → transfer → boundary**.

Callback: Revisiting the Four Failures

Earlier we asked: what single gap, shared by all four developers, made every failure possible? The answer: **no mental model of how SQL processes rows.** Now we have one.

Failure	Root cause	Fix	Capability
1 Count inflation	One-to-many join multiplied rows before COUNT	`COUNT(DISTINCT o.order_id)`	Join multiplicity awareness
2 Silent data loss	INNER JOIN drops unmatched rows	LEFT JOIN	Row-set traceability
3 Aggregate in WHERE	WHERE runs before GROUP BY	HAVING	Aggregation placement (FWGHOS)
4 Subquery drift	Same computation in two places	CTE	Composability

No mysteries, no surprises. Every failure traces to a gap in the mental model. The For-Each model, join direction, FWGHOS, and CTEs fill those gaps.

Concepts That Recur

Concept	Introduced here as	Fully developed in
For-Each / procedural trace	Nested loop model for SQL joins	Topic 6 (aggregation pipeline), Topic 9 (graph patterns), Topic 10 (traversals)
Join direction (one-to-many, many-to-one)	FK → PK preserves; PK → FK multiplies	Topic 6 (`$lookup`), Topic 10 (traversal direction)
LEFT JOIN / preserving unmatched	Keep unmatched rows, NULL-pad	Topic 6 (`preserveNullAndEmptyArrays`), Topic 9 (`OPTIONAL`)
FWGHOS pipeline	Fixed logical execution order	Topic 6 (MongoDB aggregation stages), Topic 9 (SPARQL evaluation)
CTE / single source of truth	Named intermediate result for queries	Topic 9 (SPARQL subqueries), Topic 11 (cross-model composition)
COUNT inflation	Row multiplication from joins	Topic 6 (nested array inflation), Topic 10 (path explosion)

The predict-first discipline transfers directly: different syntax, same method.

Course Map

```
Topic 1   Introduction to Data Models
Topic 2   Conceptual Modelling (ERD → relational schema)
Topic 3   Normalisation (BCNF + star schema)
    │
    ├── ▸ Topic 4  SQL Querying (For-Each, FWGHOS, CTEs)  ← you are here
    │
    └── Topic 5    Enforcement (constraints, transactions, indexes)
         ──────── relational foundation complete ────────
Topic 6   Document Model (JSON, MongoDB)
Topic 7   XML / XSD
Topics 8–9  RDF & SPARQL
Topic 10  Property Graph Model
Topic 11  Capstone: Multi-Model Architecture
```

The schema is correct (Topics 2–3). The queries are traceable (Topic 4). Next: what *protects* the schema from invalid data and what *enforces* the rules that keep it correct?

Lab Preview

Seven-part lab — predict-first query exercises against both datasets.

- **Part 1 — Count-inflation replay:** Predict row count, then verify. Fix with COUNT(DISTINCT).
- **Part 2 — LEFT JOIN vs INNER JOIN:** Predict which rows survive. Fix silent data loss.
- **Part 3 — Aggregate placement:** Move SUM from WHERE to HAVING. Trace FWGHOS.
- **Part 4 — NULL and anti-join:** Write left anti-join. Test NOT IN with NULL.
- **Part 5 — Self-join comparison:** Find employees earning more than their manager.
- **Part 6 — Chapter 3 loop-back:** Trace the 5-table BCNF query through FWGHOS.
- **Part 7 — CTE composability:** Rewrite duplicated subqueries as a CTE.

Tools: Google Colab with MySQL (`%%sql` magic)

Lab file: `04-lab-sql-querying.ipynb`